

PROJECT DOCUMENTATION

Goa Retail Sales Analytics

SQL Data Consolidation · Power BI Dashboard · EDA & Store-SKU Product Segmentation

SCOPE REVIEWED

SQL Server schema (final_sales) · Power BI report · Jupyter notebook

DOCUMENT DATE

29 August 2026

DATA VOLUME

71,76,819 rows · 17 columns · 1 consolidated table

PREPARED AS

Independent documentation & technical review

Contents

A section-by-section guide to the schema, the Power BI report, and the analysis notebook.

01 Executive Summary	1
02 Solution Architecture	1
03 Data Source & Table Schema	1
04 Power BI Dashboard	2
05 EDA & Data Cleaning — Notebook Part 1	4
06 Outlier & Anomaly Detection — Notebook Part 2	4
07 Product Segmentation & ML Model — Notebook Part 3	5

1 Executive Summary

This document reviews and describes an end-to-end retail sales analytics project built around a Goa store network's point-of-sale data. The work spans three layers, all inspected for this write-up:

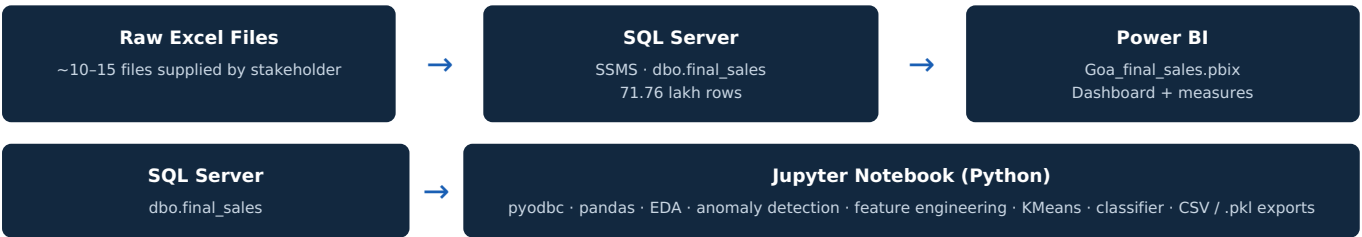
- 1. **Data engineering** — roughly 10–15 raw Excel extracts were consolidated into a single SQL Server table, `dbo.final_sales`, holding **71,76,819 rows** across **17 columns**.
- 2. **Business intelligence** — a Power BI report (`Goa_final_sales.pbix`) connects to that table and presents KPI cards, trend charts, and category/store breakdowns across a main dashboard page and a second, much larger analysis page.
- 3. **Data science** — a Jupyter notebook (`Goa_data_project.ipynb`) pulls the same table directly via `pyodbc`, performs cleaning and exploratory analysis, runs three independent outlier-detection methods with a consensus flag, and then builds a full **Store x SKU segmentation pipeline** (ABC/XYZ classification, KMeans clustering into Top/Mid/Tail tiers, and a supervised classifier trained to reproduce those tiers for new data).

Overall the pipeline is coherent end-to-end: the same grain (store, item, bill, date) flows from the raw table through both the dashboard and the notebook, and the notebook's modeling choices are documented inline with clear reasoning (e.g. why MAD z-score is used alongside IQR, why cluster labels are re-mapped by mean value rather than assumed to be in a fixed order). One item is worth double-checking before this is handed in or put into production: a page in the Power BI file is marked internally as a *tooltip* page but contains the majority of the report's visuals (see Section 4.3).

Note on naming: the SQL Server object was described as being called "Total Sales". Everywhere this document (and both the notebook and the dashboard) reference the table, the actual object name found in SSMS and used in the SQL query inside the notebook is `dbo.final_sales` — that is the name used throughout this documentation.

2 Solution Architecture

The project follows a standard "consolidate once, analyze twice" pattern — one clean table in SQL Server feeds both the BI layer and the data-science layer independently.



Layer responsibilities

Layer	Tool	What it does
Ingestion	Manual Excel → SQL Server load (SSMS)	Consolidates ~10–15 source workbooks into one table, removing Excel's row-count ceiling.
Storage	SQL Server · database (referenced in notebook as <code>Sales_New</code>)	Single source of truth table <code>dbo.final_sales</code> , queried independently by both downstream tools.
Reporting	Power BI Desktop	Interactive dashboard: KPI cards, category/store breakdowns, trend charts, slicers.
Analysis / ML	Python (Jupyter)	Cleaning, EDA, anomaly detection, store-SKU feature engineering, clustering, and a trained classifier for segment prediction.

Portability note: the notebook's connection string points at a specific local instance (`ITDSTANDBY-LAP\MSSQLSERVER01`) using a Windows trusted connection. That's fine on the machine it was built on, but it means the notebook won't run as-is on any other machine or for anyone else on the team without editing the connection cell — worth moving into a config file or environment variable before sharing.

3 Data Source & Table Schema

`dbo.final_sales` — user table, **17 columns**, created 2026-07-20 23:52:48 (per SSMS object metadata). This is the schema exactly as captured from SSMS:

ResultsMessages

	Name	Owner	Type	Created_datetime							
1	final_sales	dbo	user table	2026-07-20 23:52:48.743							

	Column_name	Type	Computed	Length	Prec	Scale	Nullable	TrimTrailingBlanks	FixedLenNullInSource	Collation	
1	STORE	smallint	no	2	5	0	yes	(n/a)	(n/a)	NULL	
2	custgroup	int	no	4	10	0	yes	(n/a)	(n/a)	NULL	
3	itemid	nvarchar	no	100			yes	(n/a)	(n/a)	SQL_Latin1_General_CP1_CI_AS	
4	itemname	nvarchar	no	200			yes	(n/a)	(n/a)	SQL_Latin1_General_CP1_CI_AS	
5	ctg	nvarchar	no	100			yes	(n/a)	(n/a)	SQL_Latin1_General_CP1_CI_AS	
6	subctg	nvarchar	no	100			yes	(n/a)	(n/a)	SQL_Latin1_General_CP1_CI_AS	
7	packsize	smallint	no	2	5	0	yes	(n/a)	(n/a)	NULL	
8	Generic_Flag	nvarchar	no	100			yes	(n/a)	(n/a)	SQL_Latin1_General_CP1_CI_AS	
9	BILLNO	nvarchar	no	100			yes	(n/a)	(n/a)	SQL_Latin1_General_CP1_CI_AS	
10	BILLDATET...	datetim...	no	8	27	7	yes	(n/a)	(n/a)	NULL	
11	SALEQTY	int	no	4	10	0	yes	(n/a)	(n/a)	NULL	
12	BILLDATE	date	no	3	10	0	yes	(n/a)	(n/a)	NULL	
13	SALEVAL	float	no	8	53	NULL	yes	(n/a)	(n/a)	NULL	
14	DISCOUNT	float	no	8	53	NULL	yes	(n/a)	(n/a)	NULL	
15	BILLS	tinyint	no	1	3	0	yes	(n/a)	(n/a)	NULL	
16	MAXSALEQ...	int	no	4	10	0	yes	(n/a)	(n/a)	NULL	
17	OFFERFLAG	nvarchar	no	100			yes	(n/a)	(n/a)	SQL_Latin1_General_CP1_CI_AS	

Column summary (business read of the schema)

Column	Type	Likely business meaning
STORE	smallint	Store code — the store network dimension used throughout the dashboard's slicers and the notebook's store-level grouping.
custgroup	int	Customer group / segment code (not used in the notebook cells reviewed — potential unused dimension).
itemid	nvarchar(100)	Item/SKU identifier — the notebook groups on (STORE, itemid) as its "store-sku" grain.
itemname	nvarchar(200)	Human-readable product name, used as the display label in every product-level chart.
ctg	nvarchar(100)	Top-level product category.
subctg	nvarchar(100)	Product sub-category.
packsize	smallint	Pack size of the item.
Generic_Flag	nvarchar(100)	Flag distinguishing generic vs. branded items (inferred from name — not exercised in the reviewed notebook code).
BILLNO	nvarchar(100)	Bill/invoice number — used via <code>nunique()</code> as the transaction-count measure.
BILLDATETIME	datetime2	Full transaction timestamp — used to derive the Morning/Afternoon/Night time-of-day segment.
SALEQTY	int	Quantity sold on the line.
BILLDATE	date	Transaction date (no time) — the main date key for daily/weekly/monthly rollups.
SALEVAL	float	Sales value / revenue for the line — the core measure in nearly every visual and calculation.
DISCOUNT	float	Discount amount applied to the line.
BILLS	tinyint	Small-range numeric field — likely a per-row bill counter/flag; its exact use wasn't exercised in the reviewed notebook cells, so confirm intent with the data owner.
MAXSALEQTY	int	Maximum recorded sale quantity for the item (reference value) — used in a Power BI bar chart alongside <code>itemname</code> .
OFFERFLAG	nvarchar(100)	Flag indicating whether the line was under a promotional offer — used as a Power BI slicer.

Precision/scale/collation for every column is visible directly in the SSMS screenshot above.

4 Power BI Dashboard — Goa_final_sales.pbix

The .pbix was inspected in Power BI's newer project-style (PBIR) internal format, which stores each page and visual as a separate JSON definition — this made it possible to enumerate every page, visual type, and field binding without opening Power BI Desktop.

4.1 Report-level settings

- **Theme:** custom theme “AccessibleTidal” layered over base theme “CY26SU07”.
- **Custom visual:** one certified custom visual, *Advance Card*, is registered and used for the KPI number tiles (an enhanced version of the built-in Card visual).
- **Background images:** two registered image resources are used as full-page backgrounds (one per page).
- **Export setting:** `exportDataMode: AllowSummarized` — viewers can export summarized visual data, not full underlying rows.

4.2 Page 1 — Main Dashboard (1280 × 720)

13 visuals: 4 KPI cards, 5 slicers, and 4 charts.

Visual type	Field(s)
Card (Advance Card)	<code>[measure] Net_sales</code>
Card (Advance Card)	<code>[measure] Total_sale_val</code>
Card (Advance Card)	<code>[measure] Item count</code>
Card (Advance Card)	<code>[measure] Discount %</code>
Card	<code>SALEQTY</code>
Area chart	<code>Month Year</code> × <code>SALEVAL</code>
Column chart	<code>ctg</code> × <code>subctg</code>
Pie chart	<code>SALEQTY</code> by <code>ctg</code>
Line & stacked column combo	<code>Year</code> × <code>[measure] Total_Bills</code>
Slicer	<code>STORE</code>
Slicer	<code>Month Year</code>
Slicer	<code>Month Name</code>
Slicer	<code>Year</code>
Slicer	<code>ctg</code>

4.3 Page 3 — Extended Analysis (2500 × 3000)

This page carries the bulk of the analytical content — **30 visuals**, including donut/clustered-bar/stacked-area/combo charts, a pivot table, 8 slicers (Year, Month Name, Date, subctg, STORE, ctg, Quarter, packsize, OFFERFLAG, Month Year), and several text boxes. It reuses most of the same KPI cards from Page 1 plus additional measures: `Avg Basket Value`, `Discount Sales Ratio`, `MoM Growth %`, `Sales This Month`, `Bills This Month`, `Total_discount`, `Total_sale_qty`.

Flagged for your attention: per the report's own page definition, this page's `type` is set to `"Tooltip"` — i.e. Power BI is configured to treat it as a hidden tooltip page rather than a normal browsable report page — yet it is also set as the file's `activePageName` (the page that opens by default) and contains far more content than the main dashboard page. Tooltip-type pages are normally hidden from the page navigation tabs at the bottom of the report. If this page is meant to be a second, fully browsable dashboard tab (which its content strongly suggests), its page type should likely be changed back to a normal page in Power BI Desktop's page-information settings — otherwise viewers opening the file may not see it in the page list at all.

4.4 Measures referenced across the report

12 named measures are used across the two pages. Their DAX formulas live inside the file's compressed internal data model, which is a binary VertiPaq store — it can't be read as text outside Power BI Desktop (e.g. via DAX Studio or Tabular Editor), so only usage, not formula logic, could be confirmed for this document:

- Net_sales
- Total_sale_val
- Item count
- Discount %
- Total_Bills
- Total_sale_qty
- Total_discount
- Avg Basket Value
- Discount Sales Ratio
- MoM Growth %
- Sales This Month
- Bills This Month

Calendar fields: the report uses `Month Year`, `Month Name`, `Year`, `Quarter`, `Day Name`, and `Date` as slicer/axis fields. None of these exist in the `final_sales` SQL schema (which only has `BILLDATE` / `BILLDATETIME`) — they're almost certainly generated inside the Power BI model itself, either via a dedicated Date table or the built-in auto date/time hierarchy on `BILLDATE`.

5 EDA & Data Cleaning — Notebook Part 1

The notebook (`Goa_data_project.ipynb`) is organized as a single narrative from raw pull through cleaning, exploration, and time-based analysis, before moving into anomaly detection and then the segmentation model.

5.1 Connecting & loading

```
connection = ("Driver={ODBC Driver 17 for SQL Server};"
              "Server=ITDSTANDBY-LAP\\MSSQLSERVER01;"
              "Database=Sales_New;"
              "Trusted_Connection=yes;")
con = pyodbc.connect(connection)
df = pd.read_sql("SELECT * FROM final_sales", con)
```

The full 71,76,819-row table is pulled directly via `pyodbc` — the notebook's own comment notes this deliberately bypasses Excel/CSV row-count limits, confirming the SQL Server hop was intentional infrastructure, not just a storage convenience.

5.2 Initial exploration

Standard first-look steps: `df.info()`, `df.shape`, and value counts by `subctg` and `ctg` to get an initial feel for category distribution.

5.3 Cleaning

Field	Treatment	Rationale (as commented in notebook)
DISCOUNT	Null → 0	A missing discount is treated as "no discount applied", not an unknown value.
SALEVAL	Null → column median	Median chosen over mean for robustness against the heavy right-skew typical of sales-value data.
Duplicate rows	Counted, then dropped	Row-level exact duplicates removed; before/after counts are printed for auditability.

5.4 Store & category analysis

Average sale value per store, average discount per item, top 5 stores by total sales value (bar/line-dot/donut views), top 10 products by category, top 10 products by quantity, and most-frequently-purchased products by distinct bill count.

5.5 Time-based analysis

- **Month-over-month growth** — sales bars with a secondary-axis MoM % growth line.
- **Year-wise sales by category** — grouped and stacked bar views.
- **Weekly trend** — line chart of sales by ISO week.
- **Time-of-day segmentation** — a custom bucketing function splits `BILLDATETIME`'s hour into Morning/Day (5–12), Afternoon (12–17), and Night (17–5), then compares total sales across the three buckets.
- **Top 10 single days** by total sales value.

A small headline "scorecard" is also built — three side-by-side panels showing Total Sales, Total Discount %, and Total Sale Quantity as big styled numbers, matching the KPI-card feel of the Power BI dashboard.

6 Outlier & Anomaly Detection — Notebook Part 2

Three independent, methodologically different techniques are applied to `SALEVAL` (and, for Isolation Forest, `DISCOUNT` and `packsize` as well), then combined into a consensus flag — a sensible way to avoid over-trusting any single method's assumptions.

Method	How it's applied	Notes
Winsorization	Top/bottom 5% capped	Produces a capped column (<code>saleval_winsorized</code>) for downstream use where extreme values shouldn't dominate, rather than a flag.
IQR rule	1.5×IQR beyond Q1/Q3	Classic Tukey fences; standard and easy to defend.

MAD z-score	Modified z-score, threshold swept 1-5, final cut at 3	Median-based, so it's more robust to the very outliers it's trying to detect than a mean-based z-score would be. The threshold sweep is a nice touch — it shows the flag count isn't wildly sensitive to the exact cutoff chosen.
Isolation Forest	<code>contamination=0.05</code> , multivariate on SALEVAL/DISCOUNT/packsize	The only multivariate method here — can catch combinations that look normal on any single field but not jointly.

A **consensus flag** is then set wherever 2 or more of the three binary methods (MAD, IQR, Isolation Forest) agree, which is a reasonable way to cut down on false positives from any single method's blind spots. Agreement counts (rows flagged by all three, by exactly two, by at least one) are printed for transparency.

Good practice observed: rather than picking one outlier method and moving on, the notebook explicitly compares all three and only flags a row when there's multi-method agreement — that's a more defensible standard than most single-method treatments, and worth keeping in whatever writeup goes to the "sir" who assigned this.

7 Product Segmentation & ML Model — Notebook Part 3

This is the largest and most technical part of the notebook: a full pipeline that classifies every (store, item) combination into a business-friendly Top / Mid / Tail tier, then trains a model that can assign that same tier to new store-SKU combinations without re-running the clustering from scratch.

Grain: one record per (STORE, itemid) pair — the notebook flags this explicitly as an assumption, and notes that dropping `STORE` from the groupby would instead give SKU-level segmentation pooled across all stores, if that's what's actually wanted.

7.1 Feature engineering

For every store-SKU pair: total sale value, total quantity, average discount, distinct bill count, active days (distinct transaction dates), first/last sale date, days since first/last sale, active ratio (active days ÷ total days in the data window), non-active days, average monthly quantity, and the coefficient of variation of monthly quantity (std ÷ mean, computed on a per-store-SKU monthly series).

7.2 ABC classification — value-based, Pareto 70/20/10

SKUs are sorted by total sale value descending, a cumulative % of total revenue is computed, and each row is labeled:

Class	Cumulative value share	Interpretation
A	0–70%	Top revenue contributors
B	70–90%	Mid-tier contributors
C	90–100%	Long tail

7.3 XYZ classification — demand variability

Class	Coefficient of variation (monthly qty)	Interpretation
X	≤ 0.5	Stable, predictable demand
Y	0.5 – 1.0	Moderately variable demand
Z	> 1.0	Highly irregular / sporadic demand

The notebook is upfront that these are textbook-convention cutoffs and may need adjusting to match whatever rubric was actually assigned.

7.4 ABC × XYZ (9-way segment) and demand buckets

ABC and XYZ are concatenated into the classic 9-cell matrix (AX, AY, AZ, BX...CZ). Separately, average monthly quantity is bucketed into five demand bands: 0–10, 10–50, 50–100, 100–200, 200+.

7.5 Ranking

Each store-SKU gets a value rank and a quantity rank (both descending, ties broken by row order), and a Final Rank computed from the average of the two, re-ranked into a clean 1..N sequence.

7.6 Top / Mid / Tail via KMeans clustering

Rather than a fixed percentile cutoff (e.g. a flat 75/15/10 split), `KMeans(n_clusters=3)` is run on four standardized features (total value, total quantity, active days, CV of quantity). Because raw cluster IDs from KMeans are arbitrary, the notebook correctly re-maps them to Top/Mid/Tail by each cluster's *mean* sale value — this avoids the common bug of assuming cluster

0 is always the "best" one.

7.7 Train / test split & model selection

An 80/20 stratified split is taken on seven features (value, quantity, active days, active ratio, CV of quantity, average discount, recency). Three candidate classifiers are compared with 5-fold cross-validation on F1-macro (a fair choice given the segment classes are likely imbalanced):

- Logistic Regression (features scaled first, since it's distance/gradient sensitive)
- Random Forest
- Gradient Boosting

The best model by CV score is then evaluated on the held-out test set (accuracy, full classification report, confusion matrix).

7.8 Hyperparameter tuning & final model

`GridSearchCV` is run over the Random Forest family (`n_estimators` : 100/200/300, `max_depth` : None/10/20, `min_samples_leaf` : 1/2/4), scored on 5-fold F1-macro. The tuned model is refit, re-evaluated on the test set, and its feature importances are plotted. Three artifacts are persisted with `joblib` so new store-SKU rows can be scored later without re-running KMeans on the whole dataset:

```
product_segmentation_model.pkl      # the tuned classifier
segment_label_encoder.pkl           # Top/Mid/Tail <-> 0/1/2 mapping
segment_feature_cols.pkl            # exact feature column order expected
```

7.9 Final export

A single table with everything — ABC, XYZ, ABC_XYZ, demand class, both ranks, Final Rank, and cluster-derived Segment — is written to `store_sku_segmentation.csv`, explicitly described as ready to feed back into Power BI or SQL.

Good practice observed: the "cluster-then-classify" pattern used here (unsupervised KMeans to define the labels once, then a supervised model trained to reproduce those labels) is a standard and defensible way to make an otherwise-expensive unsupervised step usable in production — new store-SKUs can be scored instantly against the saved model instead of re-clustering the entire table.